PRACTICAL 3 AND 4:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
pid_t = fork();
if (pid == 0){
printf("Child Process\n");
printf("PID : %d\n", getpid());
printf("PPID : %d\n", getppid());
sleep(10);
} else {
printf("Parent Process\n");
printf("PID : %d\n" getpid());
wait(NULL);
}
return 0;
}
```

```
userhasini@DESKTOP-QDSCCC4:~$ nano pid.c
userhasini@DESKTOP-QDSCCC4:~$ gcc pid.c -o pid
userhasini@DESKTOP-QDSCCC4:~$ ./pid
Parent Process
PID : 847
Child Process
PID : 848
PPID : 847
```

Progarm 4:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
int i;
for(i=0;i<3;i++){
if(fork() == 0){
printf("Child %d PID = %d\n", i+1,getpid());
return 0;
}
}
for(i=0;i<3;i++){
wait(NULL);
}
printf("All children completed.\n");
return 0;
}
```

```
userhasini@DESKTOP-QDSCCC4:~$ nano waitpid.c
userhasini@DESKTOP-QDSCCC4:~$ gcc waitpid.c -o waitpid
userhasini@DESKTOP-QDSCCC4:~$ ./waitpid
Child 1 PID = 919
Child 2 PID = 920
Child 3 PID = 921
All children completed.
```